That line of code says that if you call thing.name, you'll get Fred as a result. Crucially, the thing.name method is (by default) not touched, so whatever value the "real" method would return is not relevant; the Fred response comes from the stub, not the actual object. If thing.name is not called in the test, nothing happens.

A *mock* is similar to a stub, but in addition to returning the fake value, a mock object sets a testable expectation that the method being replaced will be called in the test. If the method is not called, the mock object triggers a test failure. You can write the following snippet to create a mocked method call instead of a stub, using expect instead of allow:

```
expect(thing).to receive(:name).and_return("Fred")
```

If you use the mock then call thing.name in your test, you still get Fred and the thing.name method is still untouched. But if you don't call thing.name in the test, the test fails with an unfulfilled-expectation error.

In other words, setting a stub on a method is passive and says, "Ignore the real implementation of this method and return this value," while setting a mock on a method is aggressive and says, "This method will return this value, and you better call the method, or else!"

Setting an expectation on whether a method is called allows you to test an object's behavior rather than its final state. Once you've stubbed a method, it makes no sense to write an assertion like this one:

```
allow(thing).to receive(:name).and_return("Fred")
expect(thing.name).to eq("Fred")
```

This test can't fail; you're just testing that the stub works as advertised. But if you do this:

```
expect(thing).to receive(:name).and_return("Fred")
```

the code is required to behave a certain way—namely, it has to call thing.name to pass the test.

A *spy* is a slight twist on a mock or stub. Generically, all that declaring a spy means is that the test-double package is watching the method. However, once you've declared a spy, you can then specify a testable expectation later in the test. Typically you would place the body of the test in between the declaration and the expectation. In RSpec, you can declare a spy the same way you would declare a stub:

```
allow(thing).to receive(:name).and_return("Fred")
# body of test
expect(thing).to have_received(:name)
```